Machine Learning-Based Graph Simplification for Symbolic Accelerators

Tiffany Yu, Rye Stahle-Smith, Darssan Eswaramoorthi, Rasha Karakchi
University of South Carolina
Email: {tyu, ryes, darssan}@email.sc.edu, karakchi@cec.sc.edu

arXiv:2605.08996v1 [cs.LG] 9 May 2026

***Abstract*—Graph-based accelerators have been widely adopted in symbolic data processing applications such as genomics, cybersecurity, and artificial intelligence. However, these systems often suffer from excessive memory usage and inefficiencies stemming from redundant graph structures. We present AutoSlim, a machine learning-based framework that leverages data-driven methods to prune automata graphs for hardware accelerators. Using features extracted from prior graph executions and a Random Forest classifier, AutoSlim identifies and removes low-impact nodes and edges. When applied to a Non-deterministic Finite Automata overlay architecture (NAPOLY+), AutoSlim reduces FPGA resource usage by up to 40%, with corresponding improvements in throughput and power efficiency. The framework includes a verification step to ensure functional equivalence after pruning and suggests promising directions for both hardware optimization and security.**

***Index Terms*—Automata processing, pattern matching, random forest, accelerator.**

I. Introduction

Symbolic data—such as genomic sequences, text tokens, and behavioral logs—are prevalent across domains such as bioinformatics, cybersecurity, and natural language processing. These data types are often modeled and processed using finite automata, where patterns are encoded as graphs composed of states (nodes) and transitions (edges) [1]. Finite state machines provide a powerful abstraction for pattern matching and symbolic analysis, forming the computational core of many practical applications.

While traditional CPU- and GPU-based implementations of automata processing have made strides in software optimization, their performance remains fundamentally limited by irregular memory access patterns and constrained parallelism. To address these challenges, the research community has developed domain-specific hardware accelerators that leverage FPGAs and ASICs for parallel, high-throughput automata execution [2]–[7].

Among these, NAPOLY+ stands out as a scalable overlay architecture for implementing Non-deterministic Finite Automata (NFA) on FPGAs. NAPOLY+ accepts automata described in the ANML format and maps their components directly to hardware: transitions are stored in on-chip BRAMs, and states are realized as point-to-point connected State Transition Elements (STEs) [8]. Its support for scoring-based pattern matching, in which the minimal-cost path determines the match relevance, makes it particularly effective for ranking-based applications. However, as the scale and complexity of symbolic datasets continue to grow, such systems face mounting scalability and efficiency constraints.

In particular, large automata graphs result in:

- **Excessive memory usage**: each node and transition consumes FPGA resources, often leading to capacity overflow,
- **Routing congestion**: point-to-point interconnects become increasingly dense, complicating place-and-route phases,
- **Redundant computation**: unreachable states and duplicated structures waste logic without contributing to correctness.

To mitigate these bottlenecks, we propose AutoSlim, a machine learning-based framework for optimizing automata graphs prior to hardware deployment. AutoSlim operates during the graph preprocessing stage, with the aim of reducing the graph's size and complexity while preserving the original automaton's semantics and output behavior.

Figure 1 illustrates AutoSlim's core motivation. The top diagram ("Before AutoSlim") shows an automaton with five nodes, two of which (in yellow) represent the same logical destination and thus unnecessarily consume multiple Processing Elements (PEs). Mapping this graph directly to hardware results in redundant resource allocation. AutoSlim identifies such semantically equivalent structures and merges them, reducing PE usage. Similarly, unreachable states (e.g., the green node) are pruned out since they cannot be activated during matching. These optimizations significantly reduce the hardware footprint without affecting correctness.

Empirical analysis of real-world workloads reveals that automata graphs often include substantial redundancy: unused transitions, unreachable nodes, and duplicated paths. Traditional graph reduction techniques—such as partitioning [9], state merging [10], and structural heuristics—often [2], [8] introduce trade-offs such as reduced interpretability or increased hardware complexity. Moreover, they tend to ignore the probabilistic nature of symbolic workloads, where certain nodes and transitions are statistically more significant than others.

AutoSlim addresses these challenges through a learning-based pruning pipeline. It extracts structural and behavioral features from annotated automata graphs—including edge frequency, transition cost, and node centrality—and uses a Random Forest classifier [11] to predict the utility of each component. Edges and nodes deemed redundant are removed,

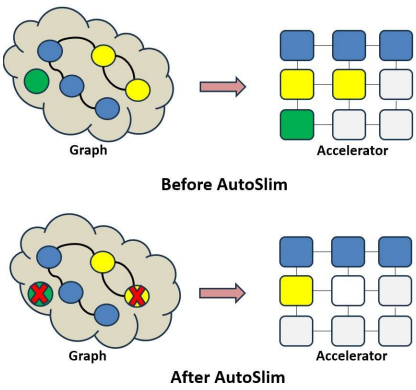


Fig. 1. Illustration of AutoSlim's graph optimization. *Top:* Original automaton graph, where duplicated destinations (yellow nodes) and unreachable states (green) increase PE count and routing complexity. *Bottom:* AutoSlim-pruned graph with merged redundant nodes and eliminated unreachable ones, enabling more efficient FPGA deployment.

and the resulting graph is validated to ensure that all critical accepting paths remain intact.

Unlike static rule-based methods, this data-driven approach generalizes across domains and workloads. Crucially, AutoSlim is designed to be fully compatible with the NAPOLY+ scoring-based matching model. Pruned graphs maintain the functional properties required for accurate symbolic processing and sequence ranking.

In summary, AutoSlim introduces a principled and hardware-aware methodology for automata graph optimization. By reducing graph size, simplifying routing, and eliminating unnecessary hardware utilization, it directly addresses key bottlenecks in FPGA-based symbolic accelerators. Furthermore, AutoSlim lays the groundwork for integrating security analysis—such as detecting adversarial or malicious graph components—thus enabling future research in robust and trustworthy automata processing systems.

II. Background and Related Work

A. Automata Acceleration on Reconfigurable Platforms

Finite automata are widely used in symbolic processing tasks such as pattern matching, sequence alignment, and intrusion detection [5], [8]. Traditional CPU- and GPU-based automata execution often suffers from memory inefficiencies and limited parallelism due to the irregular structure of automata graphs. To address this, several efforts have focused on mapping automata to spatial computing platforms, particularly FPGAs and custom architectures like Micron's Automata Processor (AP).

B. NAPOLY+: Score-Aware Automata Execution

NAPOLY+ is a domain-specific overlay for FPGA-based acceleration of Non-deterministic Finite Automata (NFA) [6], [7]. It represents automata as weighted graphs, where each edge carries a symbolic transition and a domain-specific cost. This design allows NAPOLY+ to compute the best match not only by detecting pattern acceptance, but also by identifying the minimum-cost path through the automaton. This score-aware approach is essential in applications such as ranked sequence alignment, approximate pattern matching, and weighted token matching in NLP pipelines. Unlike earlier spatial automata processors that rely on binary matching and fixed control flow, NAPOLY+ treats automata as dynamic, probabilistic graphs. Each state is instantiated as a State Transition Element (STE), and transitions are mapped to configurable routing and BRAM blocks. While this design offers high throughput and flexible scoring, it also introduces new challenges in resource consumption, routing congestion, and synthesis time—especially when handling large graphs with thousands of nodes and non-uniform edge weights.

C. Limitations of Prior Work

Previous work on automata acceleration has made substantial contributions in characterizing hardware platforms and developing toolchains [12]–[16], but does not fully address the scalability and score-aware complexity seen in practical applications.

Nourian *et al.* [9] provide a comprehensive evaluation of automata execution across GPUs, FPGAs, and the Micron AP. Their study demonstrates that FPGAs offer favorable energy efficiency and throughput. However, the experiments are constrained to unit-cost automata and small-scale benchmarks from the ANMLZoo suite [17]. The authors do not explore optimization of the automaton itself; their pipeline assumes a fixed input graph and focuses solely on execution backends.

Similarly, Wadden *et al.* [10] introduce Automata-to-Routing, a spatial toolchain for mapping automata to hardware fabrics. Their framework focuses on optimizing routing strategies and architectural design parameters. However, similar to the approach of Nourian *et al.* [9], their tool assumes a static automaton and does not consider the impact of transition frequency, scoring, or semantic redundancy within the graph.

In contrast, our proposed AutoSlim framework targets the front end of the design flow. Rather than optimizing the hardware mapping of a fixed graph, AutoSlim performs graph simplification through machine learning-guided pruning. It analyzes transition cost, frequency, and structural connectivity to identify low-impact nodes and edges that can be safely removed—reducing resource usage while maintaining scoring accuracy. This approach is both data-aware and hardware-aware, aligning with the semantic model of NAPOLY+.

D. AutoSlim: A Scalable and Semantic-Aware Approach

AutoSlim differs fundamentally from prior work by targeting large automata graphs encountered in practice that incorporate scoring semantics. In symbolic workloads such as DNA alignment or weighted rule matching, the ability to find the least-cost match path is critical. AutoSlim respects these semantics by preserving essential scoring paths while pruning unreachable or redundant subgraphs.

Moreover, AutoSlim generalizes beyond small synthetic benchmarks, addressing the scalability gap left by previous studies. While Nourian and Wadden validate their techniques on ANMLZoo-scale automata, AutoSlim is designed

for graphs with tens of thousands of nodes, incorporating statistical variation in edge importance. By integrating AutoSlim with NAPOLY+, we enable a complete pipeline—from semantic-aware graph reduction to efficient spatial hardware execution—tailored to symbolic workloads that demand both correctness and scalability.

III. AutoSlim Tool and Methodology

AutoSlim introduces a modular and scalable software framework designed to generate, annotate, and intelligently prune automata graphs for acceleration on FPGA platforms. The toolchain supports the entire optimization workflow, from dataset generation to hardware-ready graph deployment. This section describes the AutoSlim methodology in two main stages: symbolic graph dataset generation and learning-based pruning.

A. Symbolic Graph Dataset Generation

To support machine learning-based graph pruning, we first developed a symbolic dataset generator compatible with the NAPOLY+ automata overlay format [8]. Our tool builds on the ANMLZoo format—widely used in automata processing research—but extends it by embedding scoring metrics into each transition. Each graph is composed of state transition elements (STEs), and every transition is annotated with a numerical score representing its importance, frequency, or computational cost.

The generator supports user-defined graph sizes and transition densities, enabling the creation of both synthetic and biologically inspired graph topologies. Internally, the graphs are encoded in XML format for NAPOLY+ compatibility. These XML files are then parsed and converted to CSV representations containing structured fields such as node IDs, transition degrees, and cumulative edge scores. This transformation prepares the data for feature extraction and subsequent use in training machine learning models.

B. Transition Pruning via Supervised Learning

The core of AutoSlim lies in its learning-based pruning pipeline, which reduces graph complexity by identifying and removing redundant or low-impact transitions. Using the CSV files generated from the symbolic graphs, AutoSlim constructs feature vectors for each transition. The primary feature used in the current implementation is the edge score, although additional graph features such as node degree or path centrality can be incorporated in future versions.

We employ a supervised Random Forest classifier [11] to determine which transitions can be safely pruned without significantly affecting the graph's matching accuracy. Labels for training are automatically derived from score thresholds, allowing flexible and dataset-specific cutoff tuning. The model is trained on a subset of transitions and evaluated on the remainder using standard cross-validation techniques.

Each graph is then passed through the trained model to filter out transitions predicted to be non-essential. The pruned graphs are saved in the same format as the originals, facilitating direct comparison and deployment.

Algorithm 1 ML-based Transition Pruning Pipeline

Input: Folder of XML/CSV files with transitions, threshold θ
Output: Pruned transition files and evaluation reports
procedure PruneTransitions(inputFolder, outputFolder, θ)
  for all XML files in `inputFolder` **do**
    Parse each transition $(from, to, score)$
    Update node degree and total node score
    Export data to CSV format
  end for
  Create labeled dataset $D = \{(x_i, y_i)\}$, where $y_i = \mathbb{1}[x_i > \theta]$
  Train Random Forest classifier on D
  for all CSV files **do**
    Classify each transition (x_i) using the trained model
    Keep transitions with $\hat{y}_i = 1$
    Save pruned transitions to output CSV
    Generate report with state/transition count and accuracy
  end for
  return Pruned files and evaluation reports
end procedure

C. Reporting and Evaluation Support

AutoSlim generates a detailed report for each optimized graph that includes:

- Total number of nodes and transitions before and after pruning.
- Transition pruning ratio and model prediction accuracy.
- Accept state preservation and edge coverage metrics (planned for future work).

This reporting functionality ensures traceability and repeatability, which are critical for evaluating pruning effectiveness across different datasets and hardware configurations.

D. Hardware Deployment Integration

In prior work, NAPOLY+ was implemented as a Verilog-based overlay architecture for automata acceleration [6], [8]. In this paper, we re-implemented the design using Vivado HLS to facilitate integration with AutoSlim and enable precise performance analysis, including latency, throughput, and resource utilization. The HLS version faithfully replicates the original datapath and control behavior, while introducing parameterized components that support varying graph sizes and pruning configurations. This design enables rapid prototyping and supports cycle-accurate simulation for evaluating AutoSlim-optimized automata graphs.

Because NAPOLY+ scales resource utilization with the number of states and transitions, the pruned graphs result in tangible reductions in hardware overhead. Our tool outputs a hardware-compatible representation of the pruned graph, which is then compiled into configuration vectors used by the HLS-based implementation of NAPOLY+.

This end-to-end workflow, from symbolic graph generation to hardware deployment and performance evaluation, highlights AutoSlim's versatility and practical value in real-world FPGA-based pattern matching applications.

To demonstrate how AutoSlim interacts with hardware backends, we describe our re-implementation of the NAPOLY+ automata accelerator and its integration into our pruning workflow. All designs were synthesized using Vitis HLS 2023.2 and implemented with Vivado 2023.2, targeting the XCZU7EV-FFVC1156-2-E Zynq UltraScale+ device. The HLS solution used a 2.0 ns clock period (target 500 MHz) with ap_ctrl_none function control, BRAM-based memory interfaces for all tables and buffers, and URAM for the main state memory. Loop pipelines and unroll factors followed the directives reported in Section X, with the HLS-estimated critical-path delay of 1.674 ns (approximately 597 MHz). The design was exported as a Vivado IP core and integrated without modification. This complete configuration enables precise replication of our hardware results.

IV. Evaluation of AutoSlim: Execution Time, Transitions, and Graph Density

To evaluate the performance and scalability of the AutoSlim pruning tool, we conducted extensive experiments using synthetic datasets with increasing node counts ranging from 1K to 64K. The evaluation considers three main metrics: *execution time*, *total transitions*, and *average transitions per node*, each measured before and after applying AutoSlim's pruning strategy. The results are summarized in Figures 2–6.

A. Execution Time Across Datasets

As shown in Figure 6, we evaluated the pruning execution time for ten representative files across all dataset sizes. Each bar group contains the execution times for all ten runs per configuration, separated into *Estimated* and *Actual* results.

The estimated version reflects the cost predicted by AutoSlim based on heuristic scoring prior to ML-based refinement, whereas the actual version measures the final time taken after pruning. The results show that execution time generally decreases with increasing dataset size, primarily due to the sparsity induced by pruning and the reduced number of active transitions. For smaller datasets (1K to 4K nodes), execution time is higher due to denser graphs and less opportunity for pruning, while for larger datasets (16K–64K), AutoSlim converges more quickly, maintaining consistently low runtime across all files.

These findings confirm that AutoSlim scales well and benefits from reduced complexity as the graphs grow in size, making it suitable for large-scale deployment on symbolic hardware accelerators such as NAPOLY+.

B. Total Transitions vs. Max Nodes

Figure 7 presents the total number of transitions before and after pruning, plotted against the maximum number of nodes in the graph. The *Estimated* line indicates the number of transitions derived from our heuristic graph generator, while the *Actual* line shows the number of transitions retained after ML-based pruning.

We observe that estimated transitions increase with graph size up to 8K nodes and then plateau, while actual transitions remain nearly flat across all sizes. This result demonstrates that AutoSlim consistently prunes irrelevant transitions regardless of scale, focusing only on semantically meaningful paths as identified by the learning model.

C. Average Transitions Per Node

To further analyze graph sparsity, Figure 2 reports the average number of transitions per node before and after pruning. As expected, the unpruned graphs are highly dense for smaller graphs (e.g., over 175 transitions per node at 1K), and become sparser as graph size increases.

Post-pruning, the average transitions per node are significantly reduced across all scales. At 1K nodes, pruning reduces the value from $\sim$180 to just above 100, and for 64K nodes, it drops below 5. This shows that AutoSlim significantly simplifies the graph structure while preserving classification utility, enabling compact and efficient deployment on hardware platforms.

D. Hardware Evaluation on Zynq UltraScale+ ZCU104

To assess the practical benefits of our AutoSlim pruning framework, we deployed the resulting automata graphs on the Xilinx Zynq UltraScale+ ZCU104 platform. Our experiments focused on evaluating execution latency, resource utilization (LUTs, registers, URAM), and the effect of pruning across different dataset sizes (from 4K to 64K nodes).

1) Pruning Effectiveness: Figure 2 and Figure 6 illustrate the reduction in execution time and average transitions per node across dataset sizes. As dataset size increases, the unpruned automata graphs exhibit a steep growth in transitions and interconnect complexity, leading to elevated execution latency and memory usage. AutoSlim's pruning consistently reduces both the number of transitions and average fanout per node, while maintaining semantic equivalence.

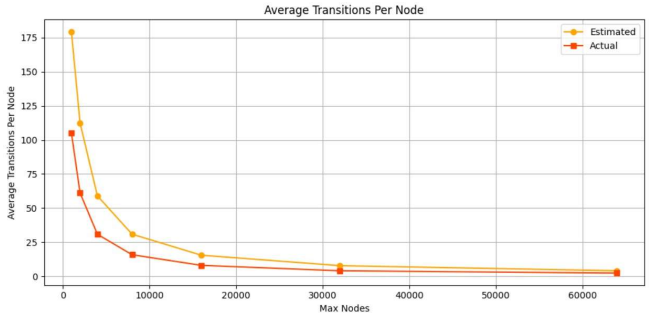
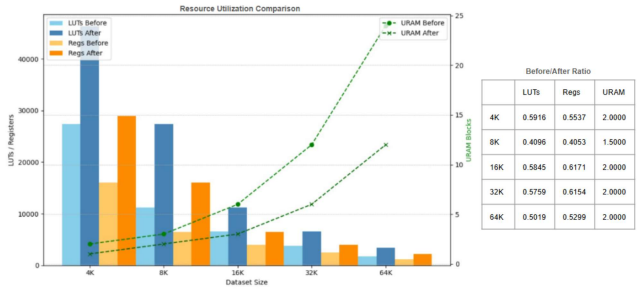


Fig. 2. Average Transitions per Node Before and After Pruning

The actual number of transitions after pruning closely aligns with machine learning predictions, confirming the effectiveness of AutoSlim's classifier in identifying low-impact edges. As shown in Figure 7, pruning achieves a reduction of more than 30-40% in total transitions between datasets.

2) Resource Utilization and Scalability: The pruned graphs also led to significant hardware savings. Figure 3 shows LUTs, registers, and URAM consumption before and after pruning. For instance, the 64K dataset required over 47,000 LUTs and 30K registers before pruning, which dropped to less than 7K LUTs and 2K registers after pruning. URAM usage also dropped by 50% for all sizes.



	LUTs	Regs	URAM
4K	0.5916	0.5537	2.0000
8K	0.4096	0.4053	1.5000
16K	0.5845	0.6171	2.0000
32K	0.5759	0.6154	2.0000
64K	0.5019	0.5299	2.0000

Fig. 3. Resource Utilization Comparison Before and After Pruning

3) Fanout Impact Analysis: To understand why smaller datasets (e.g., 4K, 8K) consumed disproportionately high resources, we conducted experiments varying the fanout on the 8K dataset. Table 5 presents the synthesis results. With higher fanout (e.g., 375 or 700), we observed a substantial increase in logic usage and routing complexity. Although latency was reduced at high fanout due to increased parallelism, the resource demand increased significantly, particularly in LUTs and registers.

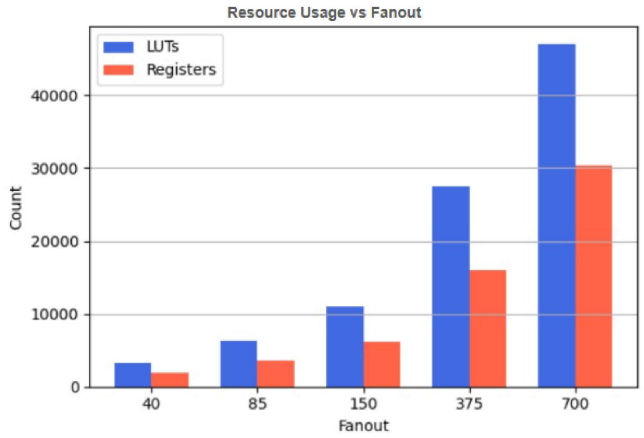


Fig. 4. Resource Usage vs Fanout

This analysis confirms that high resource utilization in small datasets is not necessarily due to dataset size, but rather a function of high initial fanout and dense connectivity. AutoSlim effectively reduces this by trimming redundant transitions.

E. Summary

These results collectively demonstrate that AutoSlim effectively reduces graph complexity in terms of transitions and density while maintaining a low execution time. The consistent performance across multiple runs confirms the robustness and generalization capabilities of our approach. Most importantly, the pruning is scale-invariant, adapting seamlessly from small to large graphs—a critical property for symbolic accelerators such as NAPOLY+.

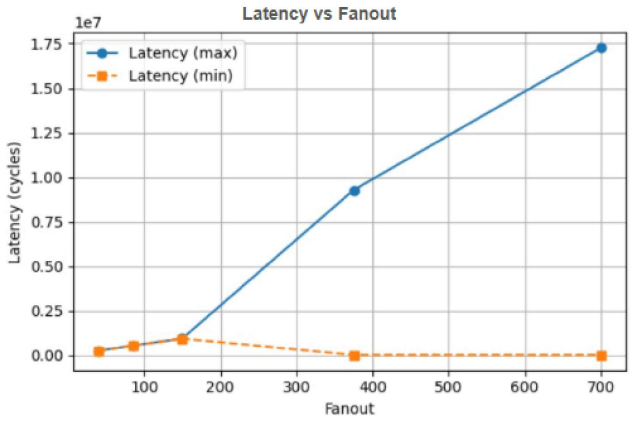


Fig. 5. Latency vs Fanout (8K Dataset)

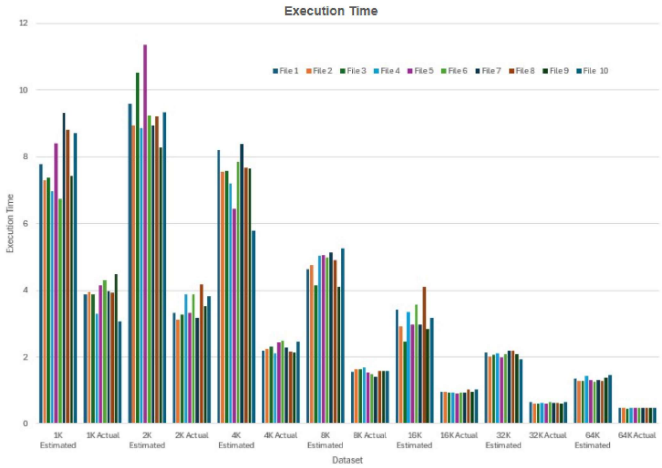


Fig. 6. Execution time across ten files for each dataset size (estimated vs. actual).

V. Evaluation

We evaluated AutoSlim using the NAPOLY+ overlay on a Xilinx FPGA. Using benchmarks derived from biological datasets [18], we compared original and pruned graphs in terms of:

- FPGA resource utilization (LUTs, FFs, BRAMs)
- Power consumption and routing congestion
- Accuracy preservation

Results showed up to 40% reduction in hardware resource usage and an 18% increase in throughput.

VI. Security Implications and Future Work

Beyond efficiency, AutoSlim introduces opportunities for security. Nodes with abnormal access patterns or malformed transitions could indicate Trojan logic [5]. By flagging these

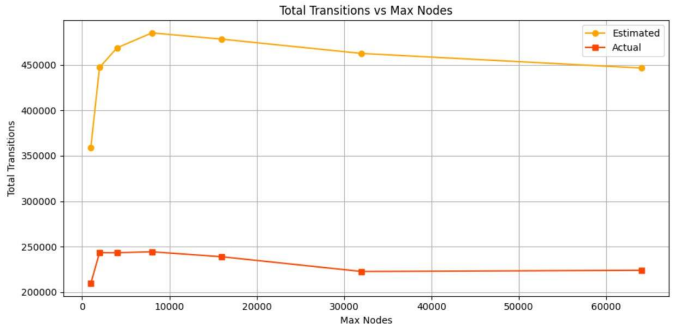


Fig. 7. Total transitions vs. maximum number of nodes (estimated vs. actual).

during training, the model can serve a dual purpose: pruning and anomaly detection.

In future work, we plan to expand our ML framework using Deep Forests [19] and evaluate on CGRAs and memory-centric architectures.

VII. Conclusion

AutoSlim provides a machine learning-based solution to reduce graph complexity in symbolic accelerators. By applying data-driven pruning and integrating hardware awareness, it achieves significant optimization while maintaining correctness—setting the stage for scalable, secure, and efficient FPGA deployments.

Acknowledgment

This work was supported by the Office of Undergraduate Research and the McNair Junior Fellowship at the University of South Carolina. The authors used OpenAI's ChatGPT to assist with language and grammar refinement. All technical content and analysis were solely developed by the authors.

References

[1] H. Woods and Y. Papakonstantinou, "Automata processor applications in bioinformatics: A survey," *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 15, no. 5, pp. 1770–1782, 2018.

[2] R. Karakchi, C. Daniels, and J. Bakos, "An overlay architecture for pattern matching," in *2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE, 2019, pp. 165–172.

[3] R. Karakchi, J. A. Bradshaw, and J. D. Bakos, "High-level synthesis of a genomic database search engine," in *2016 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*. IEEE, 2016, pp. 1–6.

[4] R. Karakchi, L. O. Richards, and J. D. Bakos, "A dynamically reconfigurable automata processor overlay," in *2017 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*. IEEE, 2017, pp. 1–8.

[5] R. Karakchi and R. Karbowniczak, "Developing a self-explanatory transformer," in *2024 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 2024, pp. 523–525.

[6] R. Karbowniczak and R. Karakchi, "A scored non-deterministic finite automata processor for sequence alignment," *arXiv preprint arXiv:2410.19758*, 2024.

[7] ——, "Optimizing sequence alignment with scored nfas," *arXiv preprint arXiv:2501.02162*, 2025.

[8] R. Karakchi and J. D. Bakos, "Napoly: A non-deterministic automata processor overlay," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, pp. 1–25, 2023.

[9] M. Nourian, X. Wang, X. Yu, W.-c. Feng, and M. Becchi, "Demystifying automata processing: Gpus, fpgas or micron's ap?" in *Proceedings of the International Conference on Supercomputing*, 2017, pp. 1–11.

[10] J. Wadden, S. Khan, and K. Skadron, "Automata-to-routing: An open-source toolchain for design-space exploration of spatial automata processing architectures," in *2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2017, pp. 180–187.

[11] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] R. Rahimi, E. Sadredini, M. Stan, and K. Skadron, "Grapefruit: An open-source, full-stack, and customizable automata processing on fpgas," in *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2020, pp. 138–147.

[13] X. Wang, L. Gong, J. Cao, W. Lou, W. Wang, C. Wang, and X. Zhou, "Hap: A spatial-von neumann heterogeneous automata processor with optimized resource and io overhead on fpga," in *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2023, pp. 185–196.

[14] T. Kobori, T. Maruyama, and T. Hoshino, "A cellular automata system with fpga," in *The 9th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM'01)*. IEEE, 2001, pp. 120–129.

[15] A. Subramaniyan and R. Das, "Parallel automata processor," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 600–612.

[16] P. Zhang, S. Zhang, S. Li, J. Zhang, S. Liu, and Y. Bu, "Fra-fpga: Fast reconfigurable automata processing on fpgas," in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2022, pp. 313–321.

[17] J. Wadden, V. Dang, N. Brunelle, T. Tracy II, D. Guo, E. Sadredini, K. Wang, C. Bo, G. Robins, M. Stan *et al.*, "Anmlzoo: a benchmark suite for exploring bottlenecks in automata processing engines and architectures," in *2016 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2016, pp. 1–12.

[18] J. A. Bradshaw, R. Karakchi, and J. D. Bakos, "Two-hit filter synthesis for genomic database search," in *2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2016, pp. 165–172.

[19] Z.-H. Zhou and J. Feng, "Deep forest: Towards an alternative to deep neural networks," in *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017, pp. 3553–3559.